CSC-100-C: *Structures*

Abanggi Kimrang S Momin

16/10/26

Why structures?

- **Problem**: Write a C program to store rollno, name and cgpa of multiple students.



1. Overwriting variables

```c
int rollno;
char name[255];
float cgpa;

rollno = 12;
strcpy(name, "John");
cgpa = 8.2;
printf("First student: %d %s %.2f\n", rollno, name, cgpa);

rollno = 14;
strcpy(name, "Ringo");
cgpa = 7.2;
printf("Second student: %d %s %.2f\n", rollno, name, cgpa);
```

What is the Drawback?

- **Overwritten** variable values are gone forever 👻



2. Unique variables

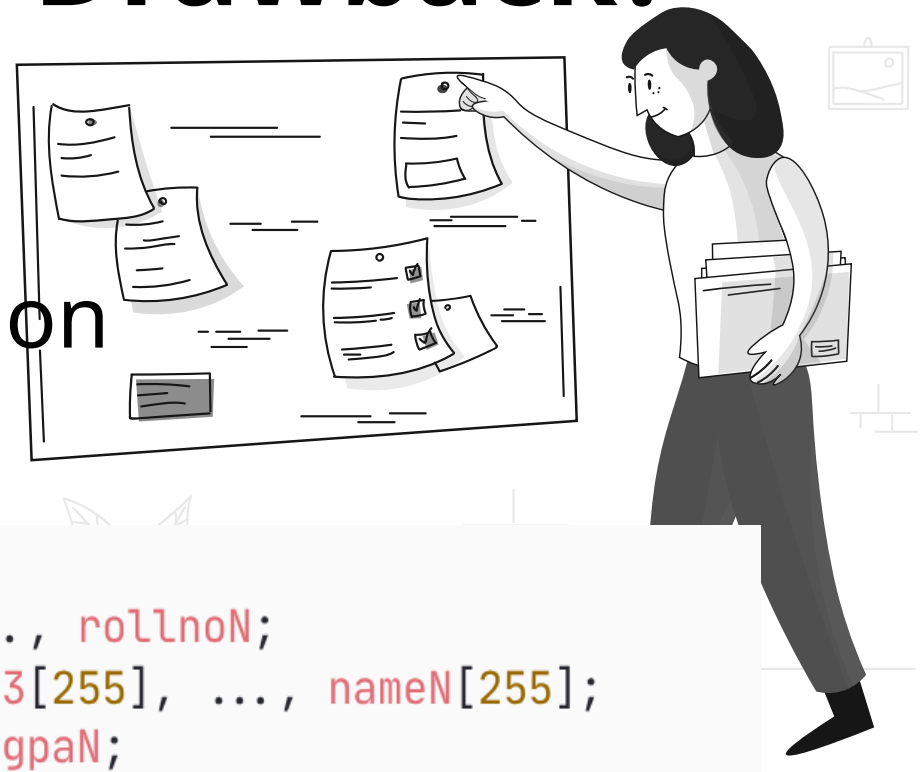
```c
int rollno1, rollno2;
char name1[255], name2[255];
float cgpa1, cgpa2;

rollno1 = 12;
strcpy(name1, "John");
cgpa1 = 8.2;
printf("First student: %d %s %.2f\n", rollno1, name1, cgpa1);

rollno2 = 14;
strcpy(name2, "Ringo");
cgpa2 = 7.2;
printf("Second student: %d %s %.2f\n", rollno2, name2, cgpa2);
```

What are the Drawback?

- **Drawback-1:**
  Repititive 👓 declaration for every student



```
int rollno1, rollno2, rollno3, ..., rollnoN;
char name1[255], name2[255], name3[255], ..., nameN[255];
float cgpa1, cgpa2, cgpa3, ..., cgpaN;
```

- **<u>Drawback-2</u>:**
  Functions that require
  N students data **require**
  N params 😞



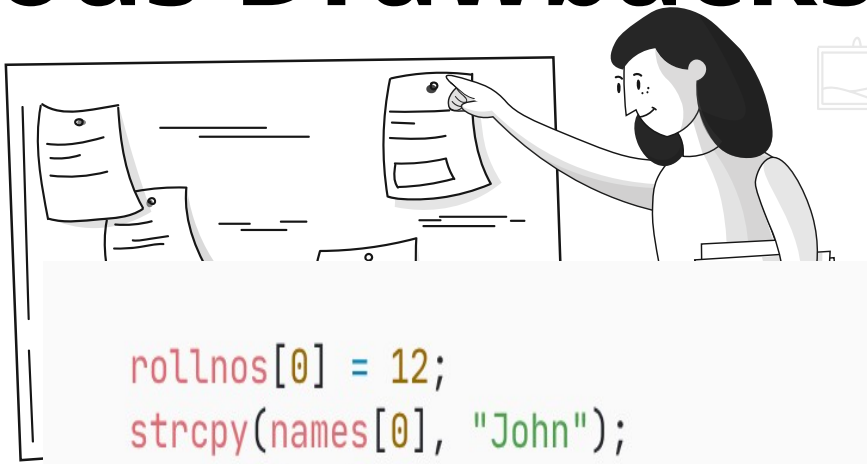
```
void displayStudentNames(char name1[], char name2[], ..., char nameN)
{
    // display logic
}
```

3. Using Arrays

```
int n = 100;     // number of students
int rollnos[n];
char names[n][255];
float cgpas[n];
```

Addressing Previous Drawbacks

- **<u>Advantage-1</u>:**
  Repititive 👓
  declaration for every student is removed 🗑️

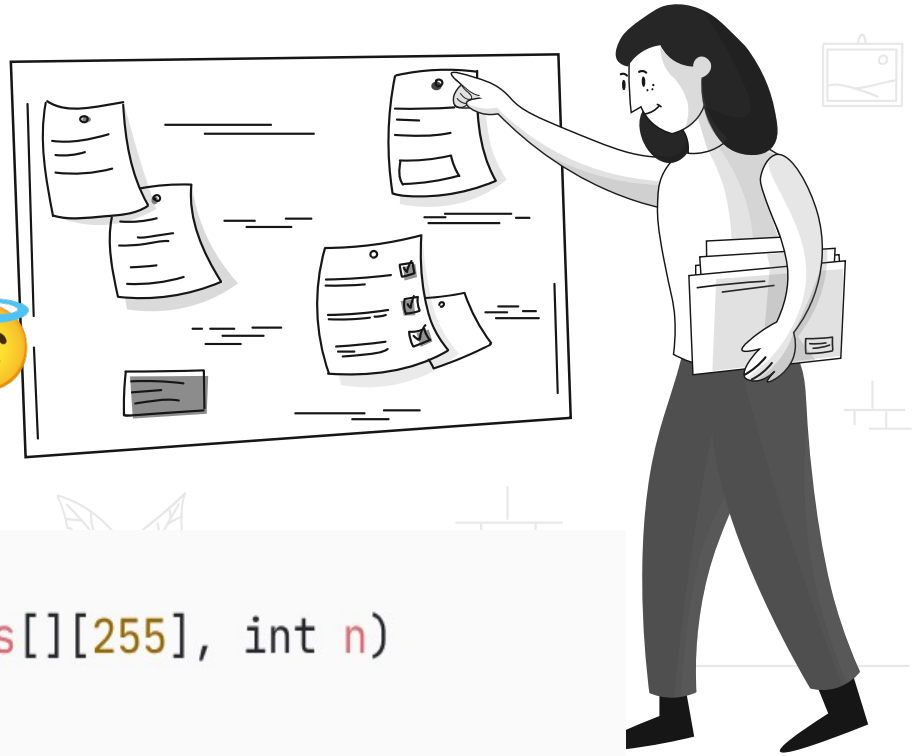


```
rollnos[0] = 12;
strcpy(names[0], "John");
cgpas[0] = 8.2;

rollnos[1] = 14;
strcpy(names[1], "Ringo");
cgpas[1] = 7.2;

...
```

- **<u>Advantage-2</u>:**
  Functions can now accept just the array 😇



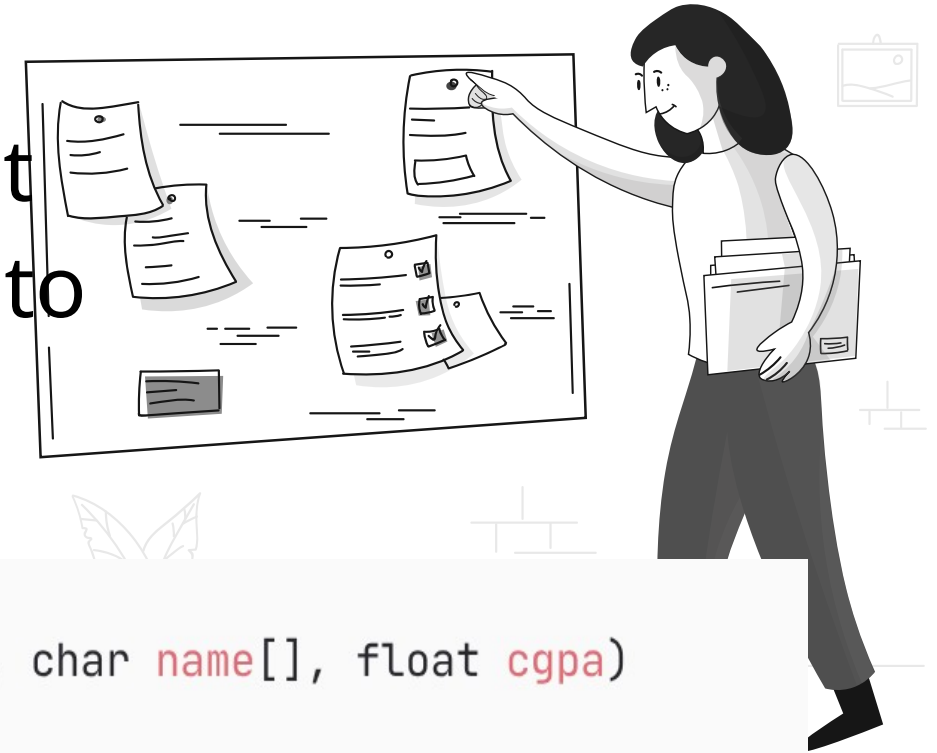
```
void displayStudentNames(char names[][255], int n)
{
    // display logic
}
```

What are the new Drawback?

- **<u>Drawback-1</u>:**
  - Data is not unitized or **not encapsulated** 📦
  - Related values are only described by **index** 👆



- **Drawback-2**:
  - Cannot send student as a single unit [👨‍🎓] to functions



```
void saveStudentInfo(int rollno, char name[], float cgpa)
{
    // saving logic
}
```

The solution is Structures!

- Structure is a **user-defined data type** that can **store related information** (even of different data types) together.
- Structure is essentially a **collection of variables under a single name**.



The syntax of Structure

- A structure is declared using the keyword `struct` **followed by the structure name**.



```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
};
```

Turning Student into Structure



Old Code

```
int rollno;
char name[255];
float cgpa;
```





New Code

```
struct student {
    int rollno;
    char name[255];
    float cgpa;
};
```

Turning Student into Structure



Old Code

```
int rollno;
char name[255];
float cgpa;
```





New Code

```
struct student {
    int rollno;
    char name[255];
    float cgpa;
};
```

Things to note

- Each variable name declared within a structure is called a ***member*** of the structure
- **The structure declaration does not allocate any memory** or consume storage space



- **The structure declaration, gives a template** that conveys to the C compiler how the structure would be laid out in the memory and also gives the details of member names



Structure variables

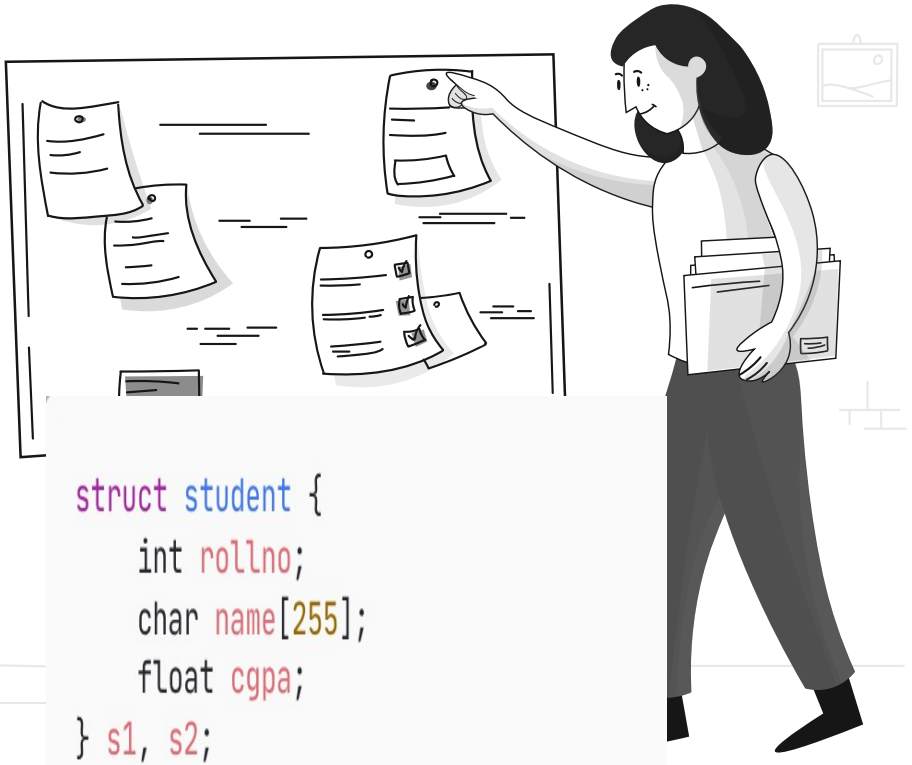
- Memory is only allocated when we declare a variable of the structure
- There are **2 ways** of declaration

  1) `struct struct-name struct-var;`

  example – **`struct student s;`**



2) At the time of structure declaration



```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
} struct-var-1, struct-var-2;
```

syntax

```
struct student {
    int rollno;
    char name[255];
    float cgpa;
} s1, s2;
```

implementation

Initializing structure variables



- Initializing a structure means **assigning some constants to the members of the structure when structure variable is created**
- Implicitly C initializes the structure
  - `int`, `float` are initialized to `0` and `0.0` and `char`, `string` to “`\0`”

- Just as declaration there are **2 ways** of initailizing a structure variable

  1) At the time of variable declaration



```
struct struct-name struct-var = {constant1, constant2, ...};
```

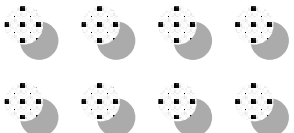
syntax





```
struct student s1 = {12, "John", 8.2};
```

implementation



2) At the time of structure declaration

```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
} struct-var = {constant1, constant2, ...};
```

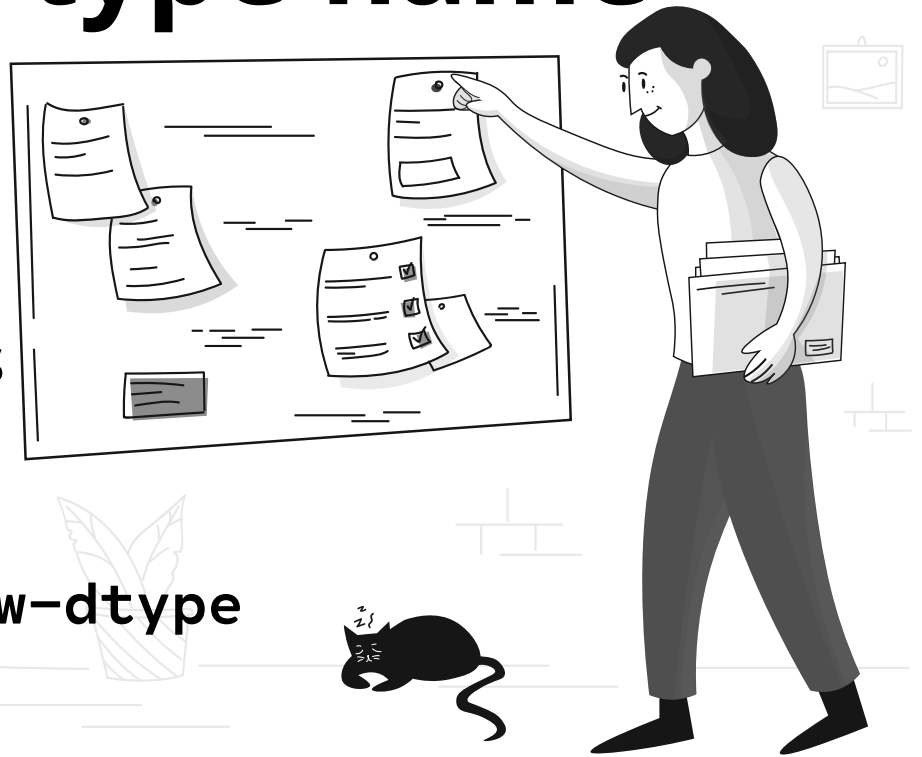
syntax



```
struct student {
    int rollno;
    char name[255];
    float cgpa;
} s1 = {12, "John", 8.2};
```

implementation

Custom data type name



- **`typedef`** keyword enables us to create new data type name from existing data type names
- Syntax

```
typedef existing-dtype new-dtype
```

- For example

```
typedef int INTEGER;
```

Using `typedef` with structures

- Syntax

```
typedef struct {
    // members
} new-dtype;
```



Re-factoring student structure



Old Code

```c
struct student {
    int rollno;
    char name[255];
    float cgpa;
};
```

```c
struct student s1 = {12, "John", 8.2};
```





New Code

```c
typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} student;
```

```c
student s1 = {12, "John", 8.2};
```

Accessing structure members



- The structure member variable can be accessed by **using a `'.'` (dot) operator**
- It can be used for both **retrieving** and **setting** the values
- Syntax:

```
struct_var.member_name
```

Retrieving member values

```c
#include <stdio.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} student;

int main()
{
    student s = {12, "John", 8.2};

    printf("Student rollno: %d\n", s.rollno);
    printf("Student name: %s\n", s.name);
    printf("Student cgpa: %.2f", s.cgpa);

    return 0;
}
```

source

```
Student rollno: 12
Student name: John
Student cgpa: 8.20
```

output

Setting member values

1) Using **assignment**

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} student;

int main()
{
    student s;

    s.rollno = 12;
    strcpy(s.name, "John");
    s.cgpa = 8.2;

    return 0;
}
```

source

Setting member values

2) Using **scanf**

```c
#include <stdio.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} student;

int main()
{
    student s;

    printf("Enter student rollno: ");
    scanf("%d", &s.rollno);

    printf("Enter student name: ");
    scanf("%s", s.name);

    printf("Enter student cgpa: ");
    scanf("%f", &s.cgpa);

    return 0;
}
```

source

Array of structures

- We can define an array **to store multiple structure variables as a unit**
- **For example** – Storing students of a particular class, Storing movies in your library etc.



- Syntax

```
struct struct_name
{
    // members
} struct_var[index];
```

or

```
struct struct_name struct_var[index];
```



Usage example

```c
struct student {
    int rollno;
    char name[255];
    float cgpa;
};

int main()
{
    struct student studs[16];

    studs[0].rollno = 12;
    strcpy(studs[0].name, "John");
    studs[0].cgpa = 8.2;

    studs[1].rollno = 14;
    strcpy(studs[1].name, "Ringo");
    studs[1].cgpa = 7.2;

    // ...

    return 0;
}
```

source

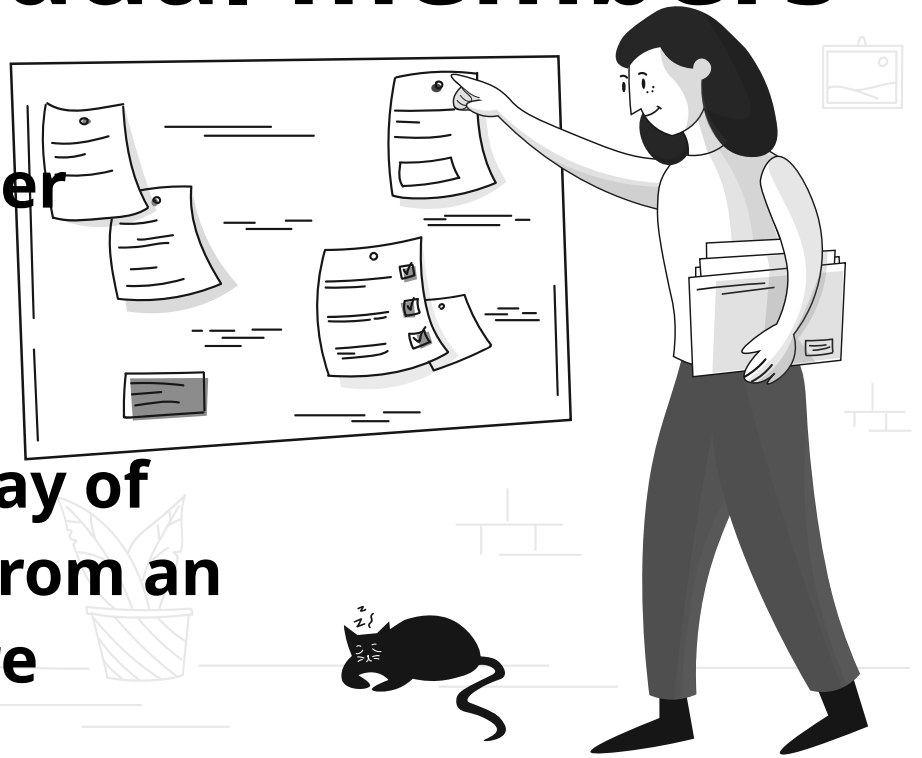
Structures & Functions



- We can **pass structures as arguments to functions** as with our normal variables
- A structure **can be passed to functions in three ways**
  1) **Passing individual members**
  2) **Passing entire structure**
  3) **Passing the address of structure**

1) Passing individual members

- We simply **refer to the member to be passed using ‘.’ (dot) operator in the function call**
- The **called function has no way of knowing if the argument is from an ordinary variable or structure variable**



Usage example

```c
#include <stdio.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} STUDENT;

void displayCGPAPerc(float);

int main()
{
    STUDENT s = {12, "John", 8.2};

    displayCGPAPerc(s.cgpa);

    return 0;
}

void displayCGPAPerc(float cgpa) {
    printf("Your percentage is %.2f%%", cgpa*10.0);
}
```

source

```
Your percentage is 82.00%
```

output

2) Passing entire structure



- When a structure is passed as an argument, **it is passed using the call by value method**, i.e., **a copy of each member of the structure is made.**
- The called function must follow the given **syntax**

```
return_type func_name(struct_name struct_var);
```

Usage example

```c
#include <stdio.h>

typedef struct
{
    int rollno;
    char name[255];
    float cgpa;
} STUDENT;

void displayStudentInformation(STUDENT);

int main()
{
    STUDENT s = {12, "John", 8.2};

    displayStudentInformation(s);

    return 0;
}

void displayStudentInformation(STUDENT stu)
{
    printf("Student rollno: %d\n", stu.rollno);
    printf("Student name: %s\n", stu.name);
    printf("Student cgpa: %f", stu.cgpa);
}
```

source

```
Student rollno: 12
Student name: John
Student cgpa: 8.20
```

output

3) Passing structure address



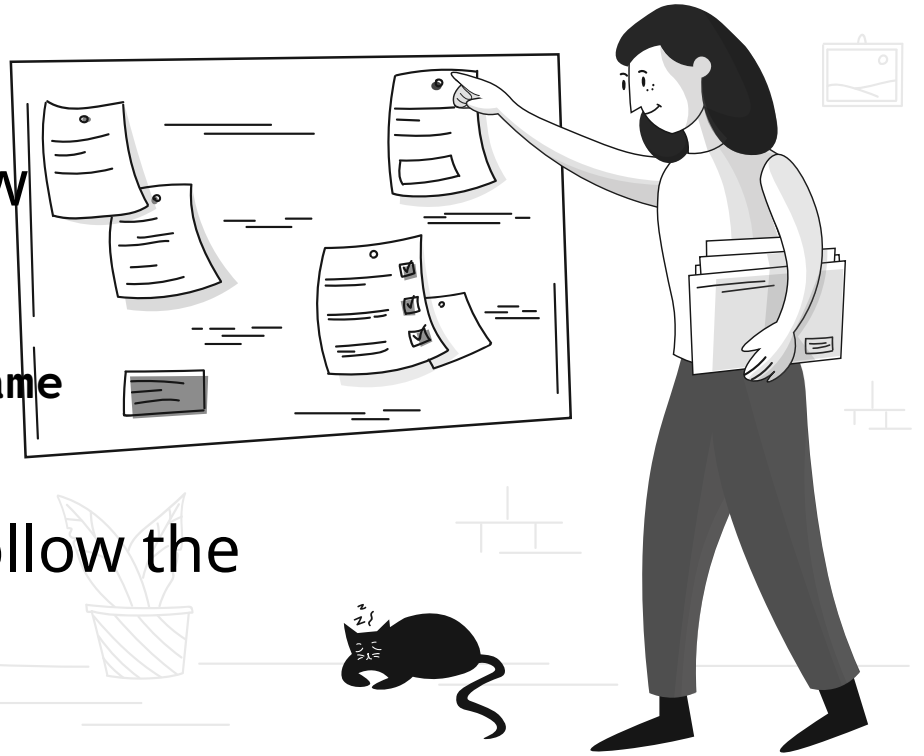
- **Passing large structures to functions using call by value is very inefficient**. Hence it is preferred to **pass the structure's address instead**
- A pointer to a structure is never itself a structure, but a variable that holds the address of a structure

- The called function must follow the general **syntax**

```
return_type func_name(struct_name
*struct_ptr)
```

- While the function call must follow the **syntax**

```
func_name(&struct_var)
```



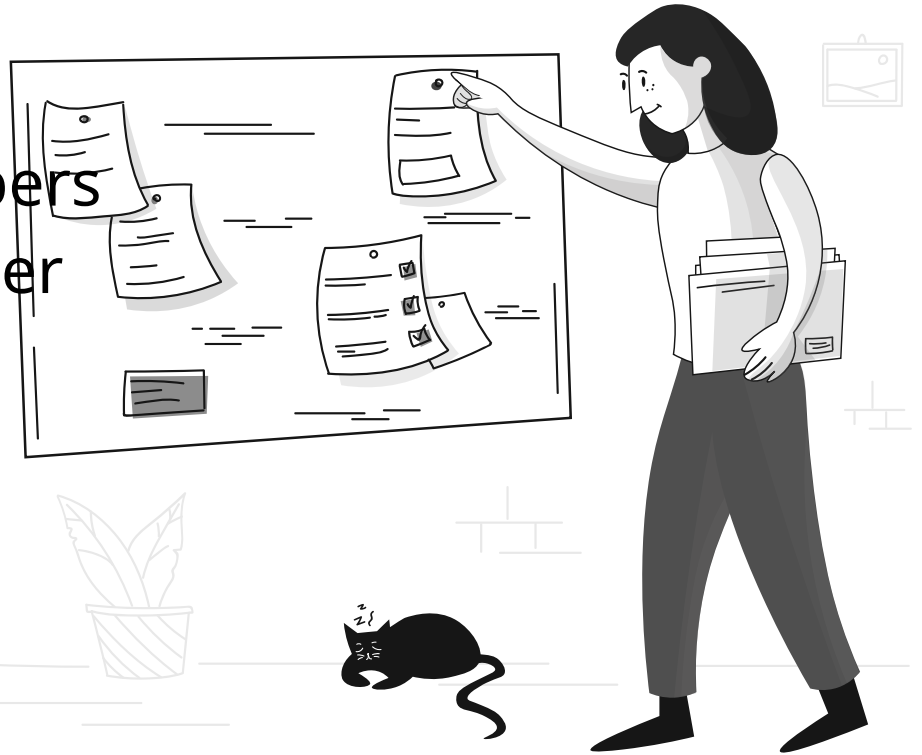
- To access the individual members from the structure we can either follow

  `(*struct_ptr).member`

  or

  `struct_ptr->member`

  - This **`->`** operator is called **“Pointing to”** operator



Usage example

```c
#include <stdio.h>

typedef struct
{
    int rollno;
    char name[255];
    float cgpa;
} STUDENT;

void collectStudentInformation(STUDENT *);

int main()
{
    STUDENT s;

    collectStudentInformation(&s);

    return 0;
}

void collectStudentInformation(STUDENT *stu)
{
    printf("Enter student rollno: ");
    scanf("%d", &stu->rollno);

    printf("Enter student name: ");
    scanf("%s", stu->name);

    printf("Enter student cgpa: ");
    scanf("%f", &stu->cgpa);
}
```

source

```
Enter student rollno: 12
Enter student name: John
Enter student cgpa: 8.2
```

output

Copying structures

- We can **assign a structure to another structure of the same type**
- Example



```c
struct student stud1 = {12, "John", 8.2};
struct student stud2;

stud2 = stud1;
```

Comparing structures

- **C doesn't allow the comparison of one structure with another**

```
stud1 == stud2     // illegal
```

- However **individual structure members can be compared**



Usage example

```c
int main()
{
    Student s1 = {12, "John", 8.2};
    Student s2 = s1;

    if (compareStudents(s1, s2) == 0)
        printf("They are the same two students");
    else
        printf("They are two different students");

    return 0;
}

/*
*   compares two strings and returns
*       0 -> if equal
*       1 -> otherwise
*/
int compareStudents(Student s1, Student s2)
{
    if (s1.rollno != s2.rollno)
        return 1;

    if (strcmp(s1.name, s2.name) != 0)
        return 1;

    if (s1.cgpa != s2.cgpa)
        return 1;

    return 0;
}
```

source

```
They are the same two students
```

output

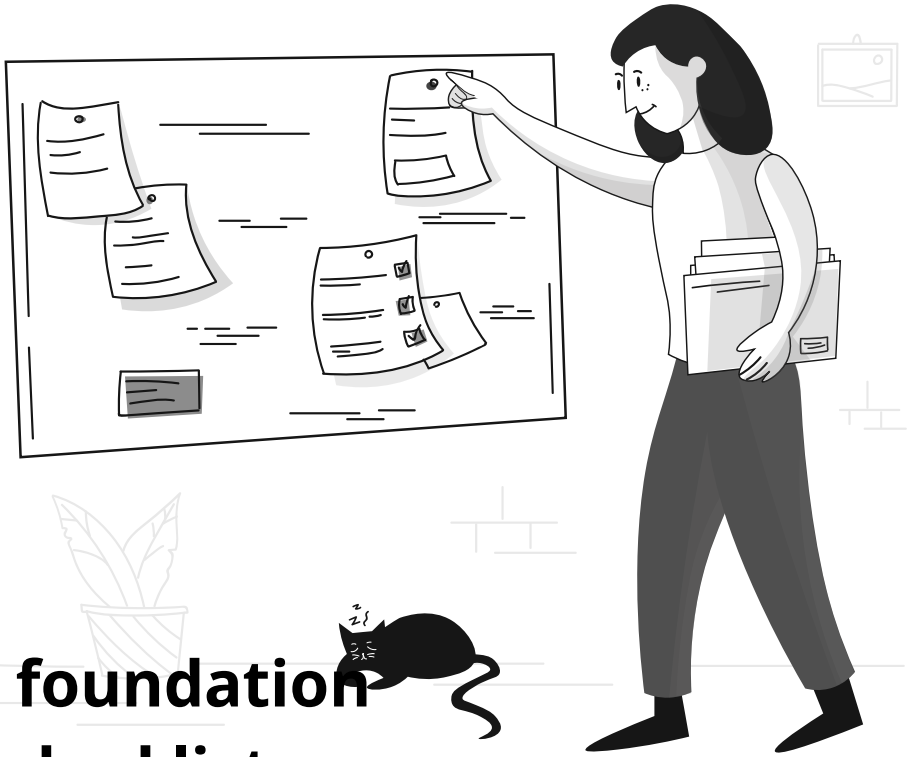
Self-referential structures

- Self-referential structures are structures that **contain a reference to the data of its same type**
- It has a **member which is a pointer to a structure of the same type**



- Usage example

```c
struct node
{
    int val;
    struct node *next;
};
```



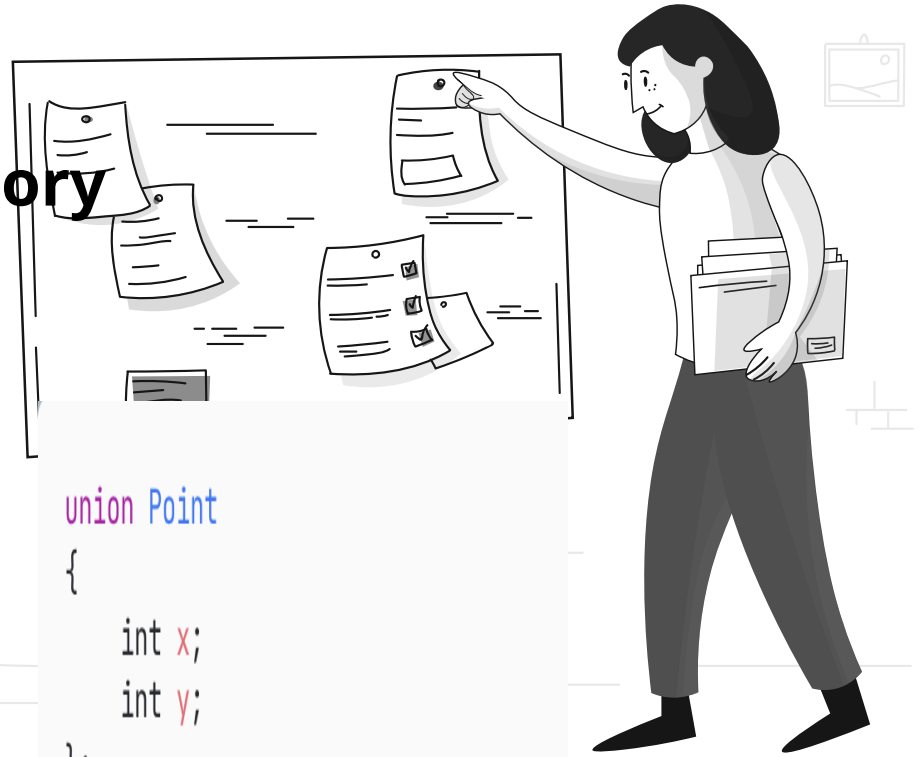
- Self-referential structure is the **foundation for data structures such as linked lists, trees and graphs**

Unions

- Unions are similar to structures with the only difference that **information can be stored in only one field at any one time**
- When a new value is assigned to a field, the existing data is replaced with the new data



- Unions are used to **save memory**
- **Syntax** for declaring a union



```
union union-name
{
    data_type var-name;
    data_type var-name;
    ...
};
```

syntax

```
union Point
{
    int x;
    int y;
};
```

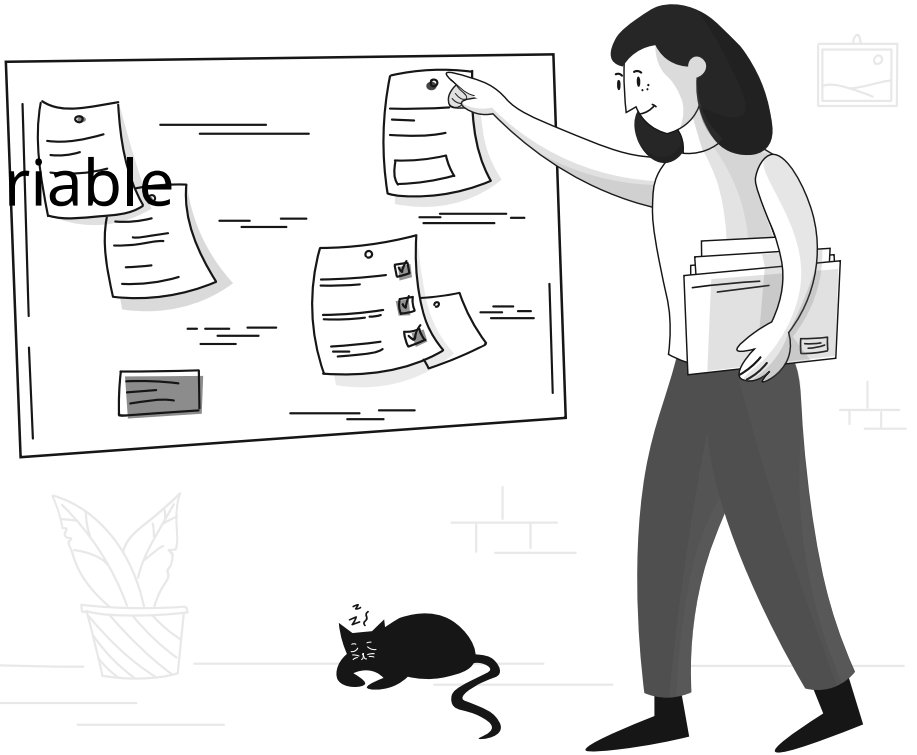
example

- The **size of a union is the size of its largest field**
- It is so that **sufficient number of bytes must be reserved to store the largest sized field**



- **Syntax for declaring** union variable

```
union union-name
{
    data_type var-name;
    data_type var-name;
    ...
} union-var;
```



or

```
union union-name union-var;
```

```
union Point p;
```

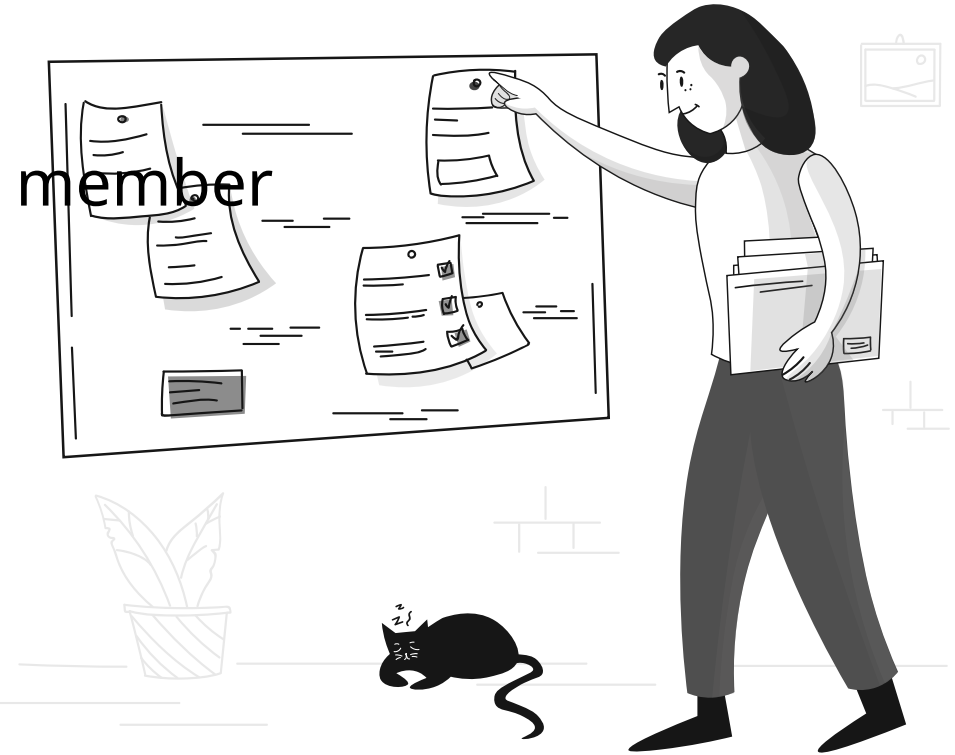
example

- **Syntax for initializing** union member

```
union union-name
{
    data_type var-name;
    data_type var-name;
    ...
} union-var = {value};
```

or

```
union union-name union-var = {value};
```



```
union Point p = {3};
```

example

Usage example

```c
#include <stdio.h>

int main()
{
    typedef union
    {
        int x;
        int y;
    } Point;

    Point p;

    p.x = 3;
    p.y = 2;

    return 0;
}
```

source

Union of structures

- A union can have **structures as its members**, in case of which, **any one of the structure can be occupying the space in memory at any given time**
- How much **memory is allocated depends entirely on it's largest structure**



Usage example

```c
typedef struct
{
    float radius;
} Circle;

typedef struct
{
    float length;
    float breadth;
} Rectangle;

typedef union
{
    Circle     cir;
    Rectangle rect;
} Shape;

int main()
{
    Shape s;

    // 1
    s.cir.radius = 2.0;

    //2
    s.rect.length = 5.0;

    // 3
    s.rect.breadth = 3.0;

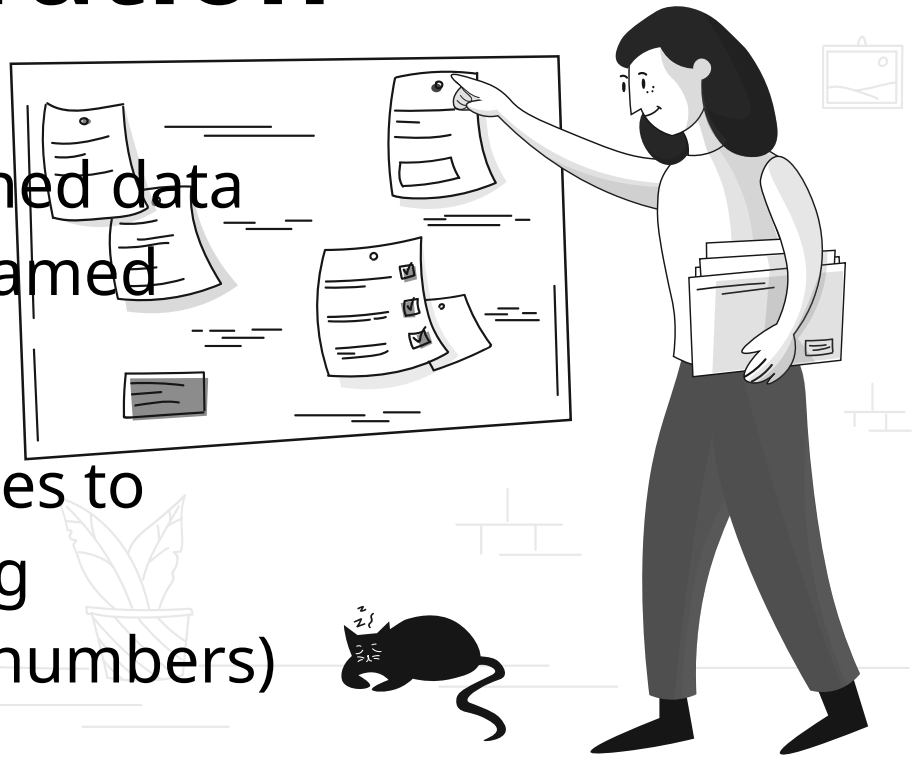
    return 0;
}

void displayAllDimensions(Shape s)
{
```

source

Enumeration

- An enumeration is a user-defined data type that consists of a set of named integer constants
- It lets us give meaningful names to integer values, instead of using unexplained numbers (magic numbers) in code

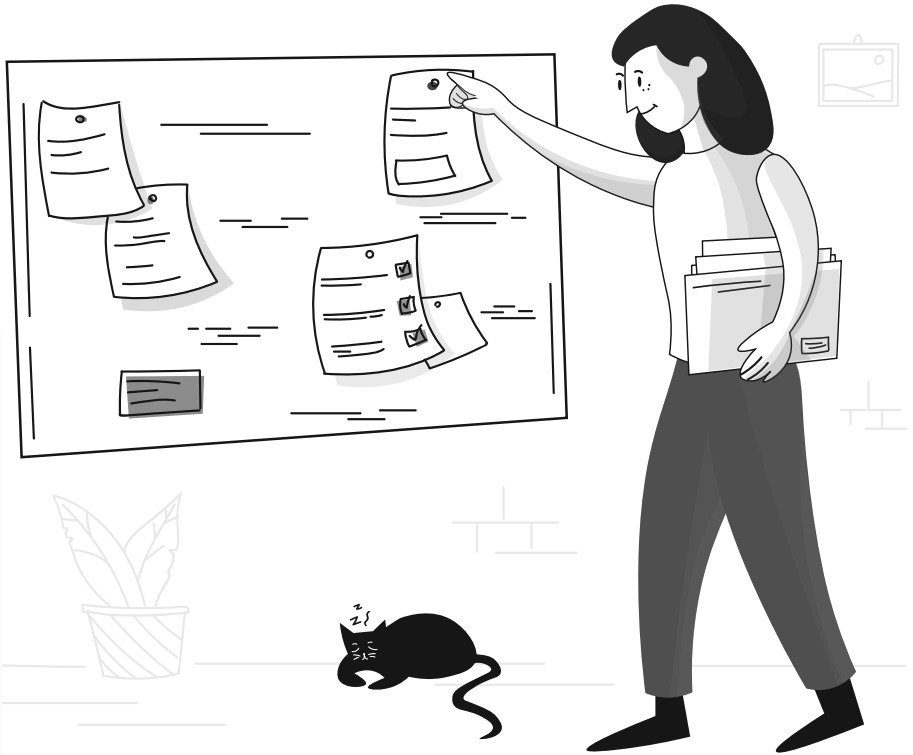


- **Syntax for declaring enum**

```
enum enum-name
{
    CONSTANT1,
    CONSTANT2,
    ...
};
```

```
enum day {
    SUN,
    MON,
    TUE,
    WED,
    THU,
    FRI,
    SAT
};
```

example



- Each constant name declared within an enumeration is called an ***enumerator*** (or ***enumeration constant***)
- By convention, **enumeration constants are written in UPPERCASE**



- **Syntax for declaring** enum variable

```
enum enum-name
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
} var-name;
```

or

```
enum enum-name var-name;
```



```
enum day today;
```

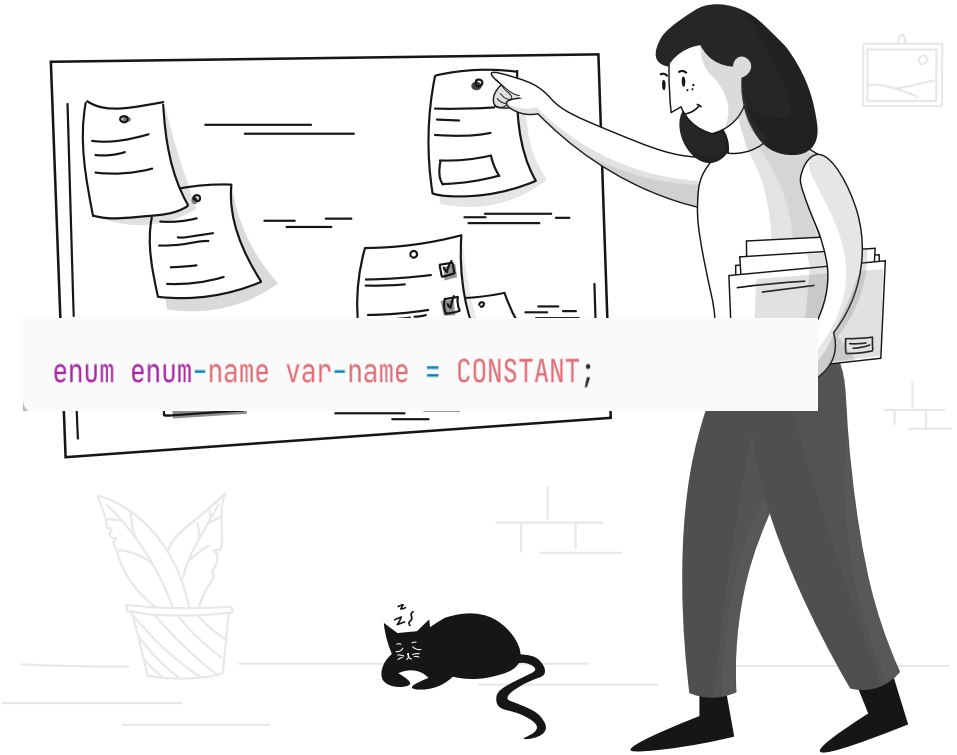
example

- **Syntax for initializing** enum variable

```
enum enum-name
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
} var-name = CONSTANT;
```

or

```
enum enum-name var-name = CONSTANT;
```



```
enum day today = WED;
```

example

Custom enumeration constants

- We can **assign our own integer values to the constants using `=`**
- A **constant without a value continues counting from the previous constant**



Usage example

```
enum http_status
{
    OK = 200,
    FORBIDDEN = 403,
    NOT_FOUND,
    SERVER_ERROR = 500
};
```

source

That’s all

